



Arm® RME System Architecture Compliance Suite

Version 3.0

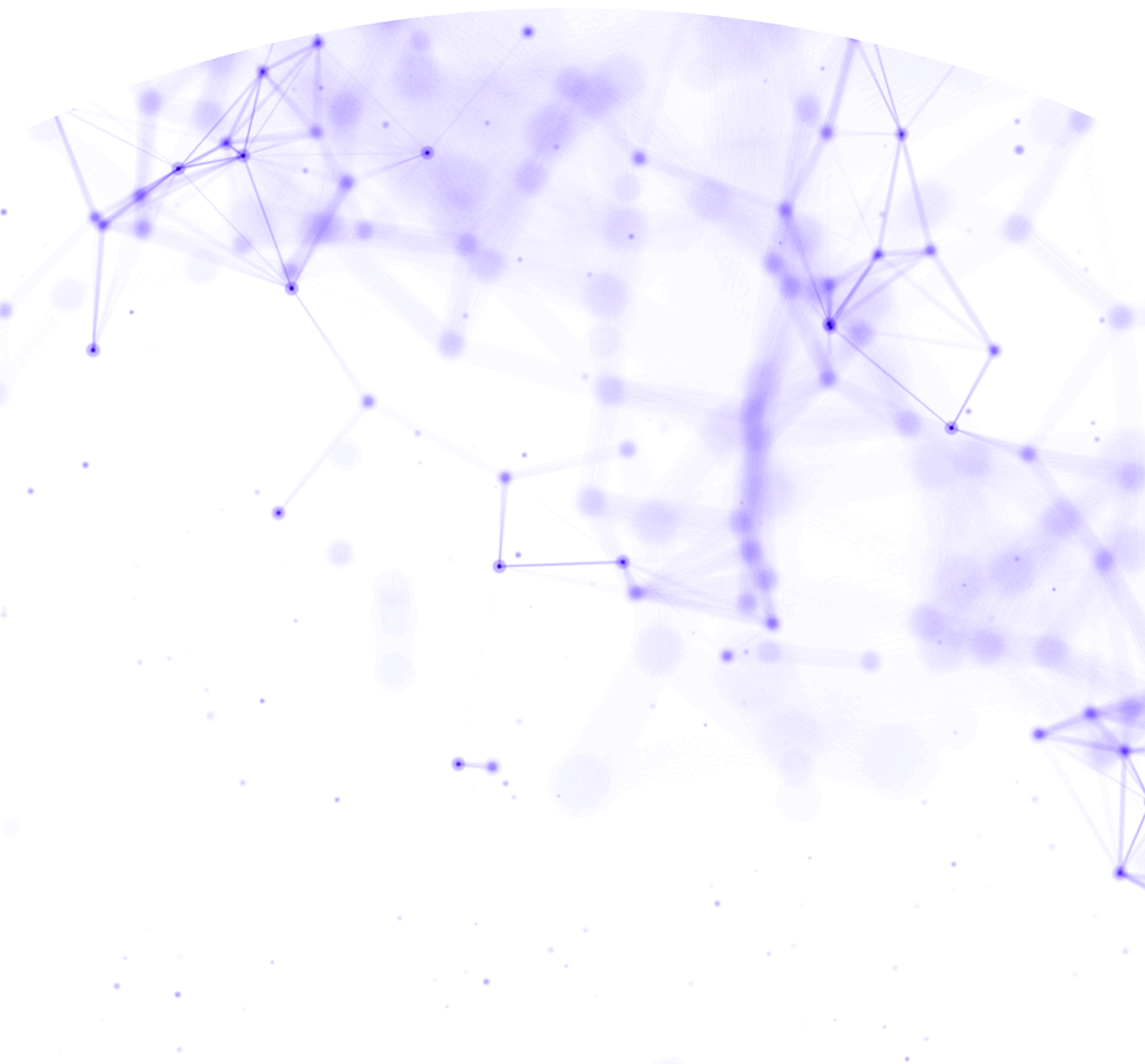
Validation Methodology

Confidential

Copyright © 2023–2026 Arm Limited (or its affiliates).
All rights reserved.

Issue 04

108004_0300_04_en



Arm® RME System Architecture Compliance Suite Validation Methodology

This document is Confidential. This document may only be used and distributed in accordance with the terms of the agreement entered into by Arm and the party that Arm delivered this document to.

Copyright © 2023–2026 Arm Limited (or its affiliates). All rights reserved.

This document is protected by copyright and other intellectual property rights.
Arm only permits use of this document if you have reviewed and accepted [Arm's Proprietary Notice](#) found at the end of this document.

This document (108004_0300_04_en) was issued on 2026-07-20. There might be a later issue at <https://developer.arm.com/documentation/108004>

The product version is 3.0.

See also: [Proprietary Notice](#) | [Product and document information](#) | [Useful resources](#)

Start reading

If you prefer, you can skip to [the start of the content](#).

Intended audience

This book is written for engineers who are designing or verifying an implementation of the Arm® Server Base System Architecture.

Inclusive language commitment

Arm values inclusive communities. Arm recognizes that we and our industry have used language that can be offensive. Arm strives to lead the industry and create change.

This document includes language that can be offensive. We will replace this language in a future issue of this document.

To report offensive language in this document, email terms@arm.com.

Feedback

Arm welcomes feedback on this product and its documentation. To provide feedback on the product, create a ticket on <https://support.developer.arm.com>.

To provide feedback on the document, fill the following survey: <https://developer.arm.com/documentation-feedback-survey>.

Contents

1. Overview to RME System ACS.....	5
1.1 Abbreviations.....	5
1.2 RME System ACS.....	6
1.3 Layered software stack.....	6
1.3.1 Compliance test software stack with UEFI application.....	8
1.3.2 Test and infrastructure guidelines.....	8
1.4 Exerciser.....	8
1.4.1 Compliance test software stack for exerciser with UEFI shell application.....	10
2. Execution flow control.....	11
2.1 Execution flow control.....	11
2.2 Test build and execution flow.....	11
2.2.1 Source code directory.....	12
2.2.2 Building the tests.....	12
2.3 EL3 ACS code integration with EL3 firmware.....	12
2.3.1 Prerequisites.....	12
2.3.2 EL3 structure and components.....	13
2.3.3 Building EL3 code.....	13
3. Platform Abstraction Layer.....	14
3.1 Overview of PAL API.....	14
3.2 PAL API definitions.....	14
3.2.1 API naming convention.....	14
3.2.2 PE APIs.....	15
3.2.3 GIC APIs.....	16
3.2.4 PCIe APIs.....	17
3.2.5 SMMU APIs.....	22
3.2.6 Exerciser APIs.....	23
3.2.7 CXL APIs.....	25
3.2.8 Miscellaneous APIs.....	26
Proprietary Notice.....	31
Product and document information.....	33
Product status.....	33
Revision history.....	33

Conventions.....35

Useful resources..... 37

1. Overview to RME System ACS

This chapter provides an introduction to the Arm® RME System Architecture Compliance Suite.

1.1 Abbreviations

The following table lists the abbreviations used in this document.

Table 1-1: Abbreviations and expansions

Abbreviation	Expansion
ACPI	Advanced Configuration and Power Interface
ACS	Architecture Compliance Suite
BDF	Bus, Device, and Function
CCA	Confidential Compute Architecture
CDA	Coherent Device Assignment
CHI	Coherent Hub Interface
CMO	Cache Maintenance Operation
CTC or C2C	Chip to Chip
CXL	Compute Express Link
DA	Device Assignment
DCM	Device Coherent Memory
DPT	Device Permission Table
ELx	Exception Level x (where x can be 0 to 3)
GIC	Generic Interrupt Controller
HCM	Host Coherent Memory
IDE	Integrity and Data Encryption
ITS	Interrupt Translation Service
IOMMU	Input-Output Memory Management Unit
KM	Key Management
LPI	Locality-specific Peripheral Interrupt
MEC	Memory Encryption Context
MECID	Memory Encryption Context Identifier
MPAM	Memory System Resource Partitioning and Monitoring
MSI	Message-Signaled Interrupt
PAL	Platform Abstraction Layer
PMU	Performance Monitoring Unit
PCIe	Peripheral Component Interconnect Express
PE	Processing Element
PoC	Point of Coherence
PoE	Point of Encryption
PoPA	Point of Physical Aliasing
PSCI	Power State Coordination Interface
RAS	Reliability, Availability, and Serviceability
RCiEP	Root Complex integrated End Point
RID	Requester ID

Abbreviation	Expansion
RME	Realm Management Extension
RMSD	Realm Management Security Domain
RMM	Realm Management Monitor
SMC	Secure Monitor Call
SMMU	System Memory Management Unit
SoC	System on Chip
SSD	System Security Domain
TEE	Trusted Execution Environment
TDI	TEE Device Interface
TDISP	TEE Device Interface Security Protocol
UART	Universal Asynchronous Receiver and Transmitter
UEFI	Unified Extensible Firmware Interface
VAL	Validation Abstraction Layer
VDM	Vendor Defined Message

1.2 RME System ACS

Realm Management Extension Architecture is an extension to the Armv9-A profile architecture. It adds the following features:

- Two additional Security states, Root and Realm.
- Two additional physical address spaces, Root and Realm.
- The ability to dynamically transition memory granules between physical address spaces.
- Granule Protection Check mechanism.

With the other components of the Arm CCA, RME enables support for dynamic, attestable, and trusted execution environments (Realms) to be run on Arm architecture.

The RME architecture defines the set of hardware features and properties that are required to comply with the Arm CCA architecture. Implementations compliant with the RME System architecture must conform to the behavior described in the [Arm® Realm Management Extension \(RME\) System Architecture specification](#). The RME ACS is a comprehensive open source test suite which ensures that the implementations compliant with the RME System Architecture must comply with the behaviors described in the RME System Architecture specification for achieving confidential computing execution environment.

For more information on ACS coverage being incomplete, non-exhaustive and non-comprehensive, see the *RME System Architecture Compliance Suite Scenario Document*.

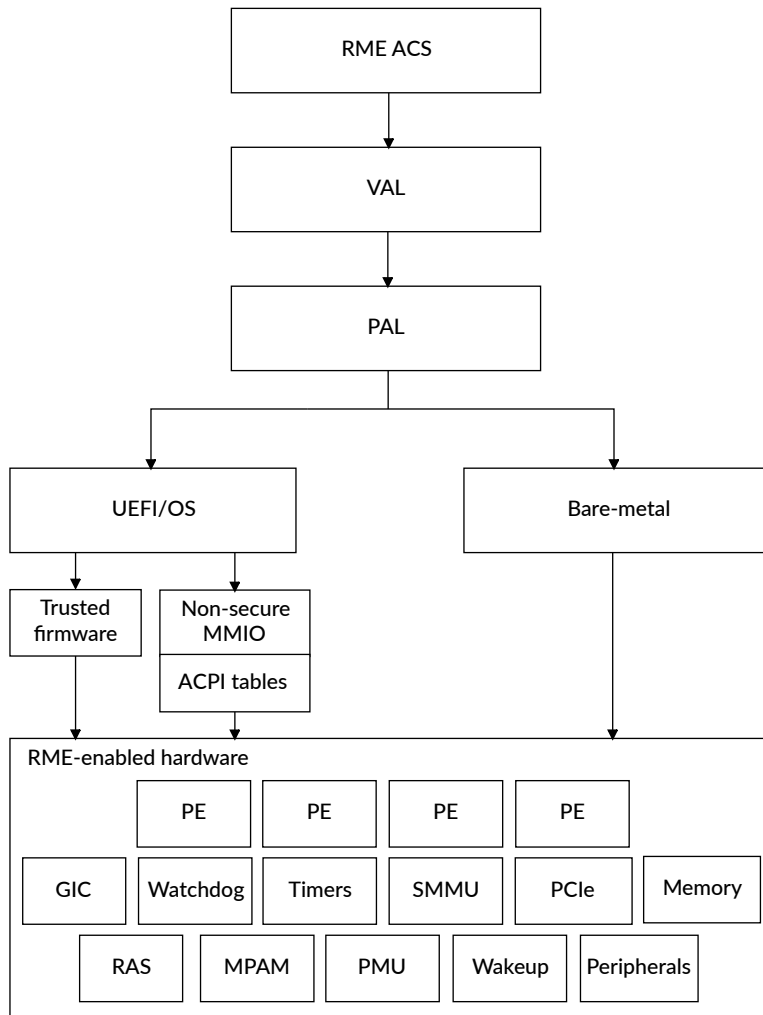
1.3 Layered software stack

Compliance tests use the layered software stack approach to enable porting across different test platforms.

The layered stack contains:

- Test suite

- Validation Abstraction Layer (VAL)
- Platform Abstraction Layer (PAL)

Figure 1-1: Layered software stack

The following table describes the different layers of a compliance test.

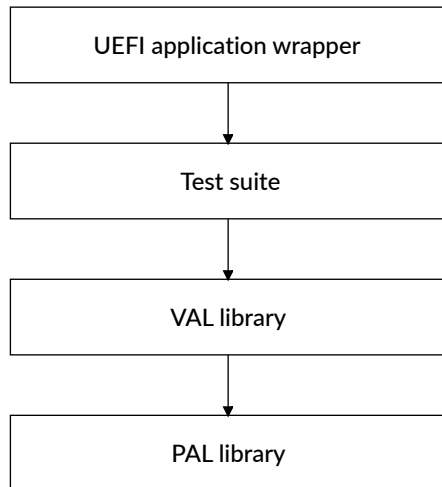
Table 1-2: Compliance test layers

Layer	Description
test_pool	Collection of targeted tests that validate the compliance of the target system. These tests use interfaces that are provided by the VAL.
VAL	Provides a uniform view of all the underlying hardware and test infrastructure to the test suite.
PAL	Contains C-based Arm-defined APIs that you can implement. It abstracts features whose implementation varies from one target system to another. Each test platform requires a PAL implementation of its own. PAL APIs are meant for the compliance test to reach or use other abstractions in the test platform such as the UEFI infrastructure and bare-metal abstraction.

1.3.1 Compliance test software stack with UEFI application

The following figure is an example of the compliance test software stack interplay with UEFI shell application.

Figure 1-2: Software stack UEFI shell application



1.3.2 Test and infrastructure guidelines

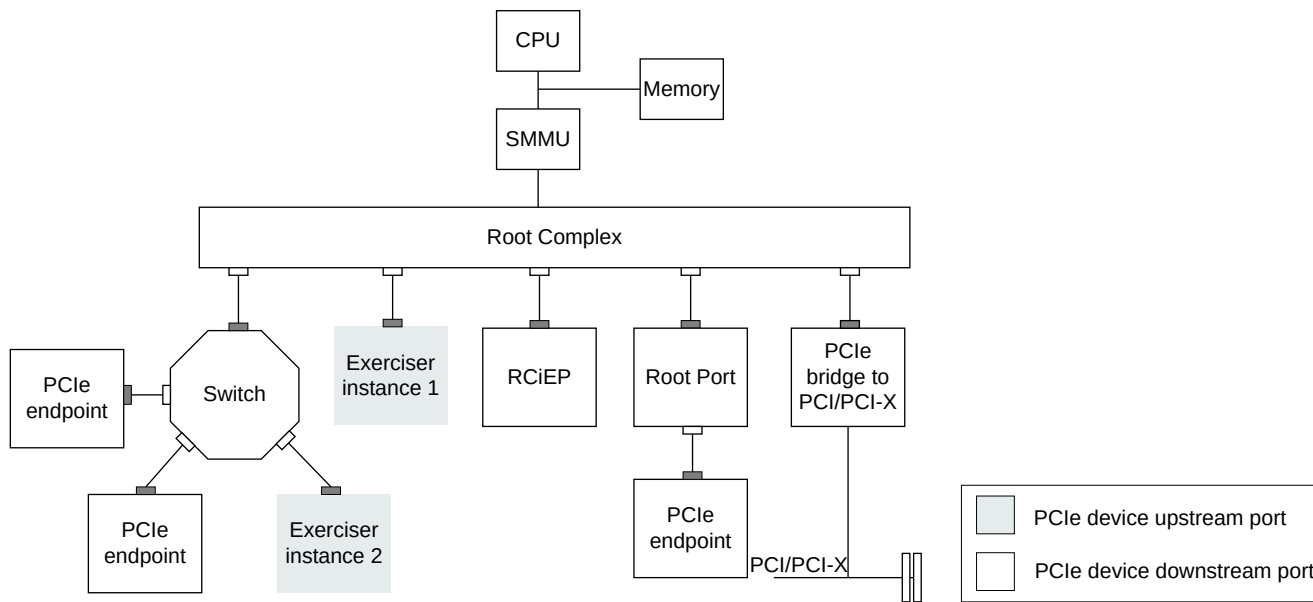
The coding guidelines followed for the implementation of the test suite are listed as follows:

- All the tests call VAL APIs.
- VAL APIs might call PAL APIs depending on the requested functionality.
- A test does not directly interface with PAL functions.
- The test layer does not need any code modifications when porting from one platform to another.
- All the platform porting changes are limited to PAL.
- The VAL may require changes if there are architectural changes impacting multiple platforms.

1.4 Exerciser

Exerciser is a PCIe endpoint device that can be programmed to generate custom stimuli for verifying the RME compliance of PCIe IP integration into an Arm SoC. The stimulus is used in verifying the compliance of PCIe functionality like IO coherency, snoop behavior, address translation, PASID transactions, DMA transactions, MSI, and legacy interrupt behavior.

The following figure shows a PCIe hierarchy consisting of various endpoints, switches, and bridges.

Figure 1-3: Exerciser in an SoC

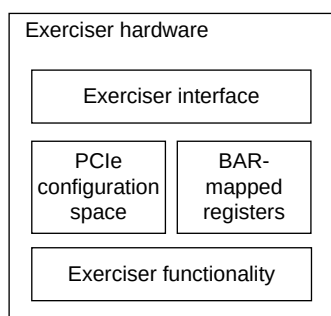
The figure shows two instances of the exerciser that are present in the system. Instance 1 is connected directly to the Root Complex as a RCiEP and instance 2 is connected to the downstream port of a switch as a PCIe endpoint device.



The number of exercisers instantiated is platform-specific. To achieve higher coverage, Arm recommends that you present multiple exercisers to the ACS.

To generate custom stimuli, the exerciser must provide functionality to configure interrupt and DMA attributes, trigger them, and know the status of these operations, the details of which are **IMPLEMENTATION DEFINED**. This can be done by providing a set of BAR-mapped registers and writing specific values to trigger the necessary operations.

The following figure shows the reference implementation of exerciser hardware.

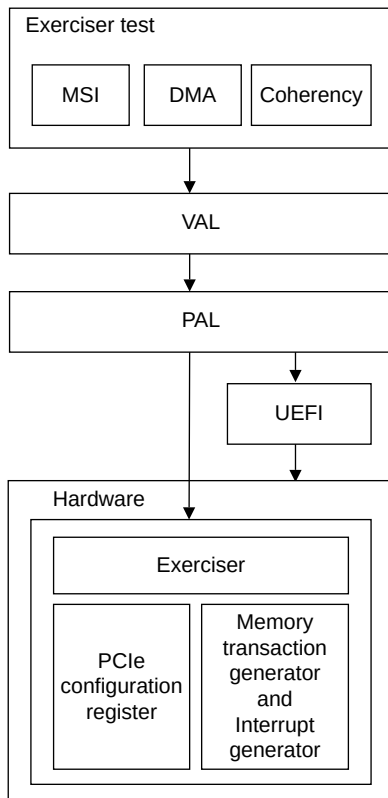
Figure 1-4: Reference implementation of exerciser hardware

1.4.1 Compliance test software stack for exerciser with UEFI shell application

The exerciser validates PCIe devices that only access non-secure address space. The exerciser PCIe configuration space is accessed using UEFI or MMIO APIs and exerciser functionality like interrupt generation and DMA transactions can be accessed using exerciser APIs.

The following figure shows the compliance test software stack for exerciser with UEFI shell application.

Figure 1-5: Exerciser with UEFI shell application



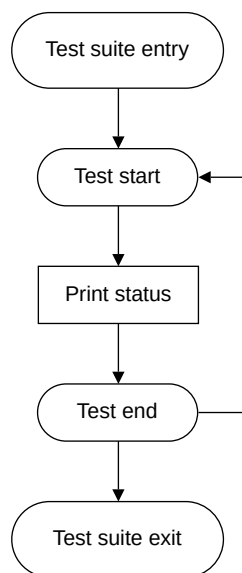
2. Execution flow control

This chapter describes the execution flow control used for RME System ACS.

2.1 Execution flow control

The following figure describes the execution flow control of the compliance suite.

Figure 2-1: Execution flow control



The process that is followed for the flow control is:

1. The execution environment such as the UEFI shell, invokes the test entry point.
2. Start the test iteration loop.
3. Print status during the test execution as required.
4. Reboot or put the system to sleep as required.
5. Loop until all the tests are completed.

2.2 Test build and execution flow

This section describes the source code directory structure and provides references for building the tests.

2.2.1 Source code directory

The following describes all the directories in the RME System ACS.

docs	Documentation.
baremetal_app	Reference bare-metal application source to call into the test entry point.
ext	This repository includes third-party or external dependencies (For example: libspdm, spdm-emu, and headers).
prebuilt_images	Pre-generated binaries or images that run the suite.
pal_el3	Contains EL3 code abstraction for val_el3.
pal_baremetal	Platform reference code for Bare-metal.
pal_uefi	Platform code targeting UEFI implementation.
test_pool	Test case source files for the test suite.
tools	Consists of scripts written for this suite and SysARCI for partner input.
uefi_app	UEFI application source to call into the tests entry point.
val	Common code that is used by the tests. Makes calls to PAL as necessary.
val_el3	Common EL3 code that is used by the tests. Makes calls to pal_el3 as necessary.

Inputs from the partners must be provided in the following files:



1. Use the `pal_el3/<plat>/include/pal_el3_config.h` file for EL3 configurations.
2. Use the `prebuilt_images/<plat>_config.ini` file. Refer the existing files for base-revc and rdrv3. If you do not use `parser.efi` for configuration updates, provide inputs in the following file:
`platform/pal_<platform_name>/include/pal_override_<platform_name>.h`

2.2.2 Building the tests

The build steps for the compliance suite to be compiled as a UEFI shell application are available in the [README](#).

2.3 EL3 ACS code integration with EL3 firmware

Install the software stack in which trusted firmware package is available. It is the reference implementation of secure world software for Arm A-Profile architectures, including an Exception Level 3 (EL3) Secure Monitor.

2.3.1 Prerequisites

- When `smc` with `SMC_Fid` is executed from Non-secure EL2, make sure you branch to the ACS User SMC handler function which is predefined in `ACK`.
- 4KB/16KB/64KB shared memory between EL3 and EL2 to share data to reflect the needs and requirements of the test clearly at both the levels.
- 2MB of memory is flat-mapped as Root Access PAS and is used for MMU page tables in EL3 and SP_EL3 by the ACS User SMC handler function in EL3.

- 2MB of memory is flat-mapped as Realm Access PAS and is used for Realm SMMU tables and Realm page tables.
- 2MB free PA used only in the tests as PA.
- 512MB unused VA space (within 48bits) used in the tests as VA.
- 4KB of non-volatile memory used in the reset tests.



See the `val_el3/README.md` for the ACS SMC handler function and `SMC_FID`.

2.3.2 EL3 structure and components

Below are the EL3 components and their descriptions.

val_el3/ :

EL3-side validation support code (runs in TF-A/EL3).

- SMC dispatcher: `val_el3_plat_smc_handler.c`
- Exception or ACK handler: `val_el3_exception.c`, `include/val_el3_exception.h`
- Page tables (MMU or GPT) helpers: `val_el3_pgt.c`, `include/val_el3_pgt.h`
- Shared memory and EL3 allocator: `val_el3_memory.c`, `include/val_el3_memory.h`
- Security helpers: `val_el3_security.c`, `include/val_el3_security.h`
- SMMU helpers: `val_el3_smmu.c`, `val_el3_smmu.h`
- MEC helpers: `val_el3_mec.c`, `val_el3_mec.h`
- PE or system register helpers: `val_el3_pe.c`, `val_el3_pe.h`
- Root watchdog helpers: `val_el3_wd.c`, `val_el3_wd.h`
- Debug or log wrappers: `val_el3_debug.h`

pal_el3/ :

Platform abstraction layer used by `val_el3/`.

- PAL APIs: `pal_el3.h`
- Platform config defaults: `pal_el3_config.h`
- Reference PAL impl or stubs: `pal_el3.c`
- EL3 logging backend: `pal_el3_print.h`

2.3.3 Building EL3 code

To generate binary file for EL3 code, follow the build steps in `README` of `val_el3`.

3. Platform Abstraction Layer

This chapter provides an overview of PAL API and its categories.

3.1 Overview of PAL API

The PAL is a C-based, Arm-defined API that you can implement.

Each test platform requires a PAL implementation of its own. The PAL APIs are meant for the compliance tests to reach or use other abstractions in the test platform such as the UEFI infrastructure and Linux OS modules. PAL implementation can also be bare-metal code.

The reference PAL implementations are available in the following directories:

- UEFI
- PAL_EL3



The PAL bare-metal reference code provides a reference implementation for a subset of APIs. The current version of the repository contains the reference code for creation of information tables like PE, GIC, timer, and watchdog. Additional code must be implemented to match the target SoC implementation under test.

3.2 PAL API definitions

The PAL API contains APIs that:

- Are called by the VAL and implemented by the platform.
- Begin with the prefix `pal`.
- Have a second word on the API name that indicates the module which implements this API.
- Have the mapping of the module as per the table below.
- Create and fill structures needed as prerequisites for the test suite, named as `pal_<module>_create_info_table`.

3.2.1 API naming convention

The PAL API interface `<module>` names are mapped as shown in the following table.

Table 3-1: Modules and corresponding API names

Module	API name
DA	da
RME	rme
GIC	gic
SMMU	smmu

Module	API name
Legacy System	legacy
DPT	dpt
MEC	mec
CXL	cxl
CDA	cda
TDISP	tdisp

3.2.2 PE APIs

The following APIs provide the information and functionality required by the test suite that accesses features of a PE.

Table 3-2: PE APIs and their descriptions

API name	Function prototype	Description
get_num	<code>uint32_t pal_pe_get_num();</code>	Returns the number of PEs in the system.
create_info_table	<code>void pal_pe_create_info_table(PE_INFO_TABLE *PeTable);</code>	Gathers information about the PEs in the system and fills the info_table with the relevant data.
call_smc	<code>void pal_pe_call_smc(ARM_SMC_ARGS *args);</code>	Abstracts the smc instruction. The input arguments to this function are x0 to x7 registers filled in with the appropriate parameters.
execute_payload	<code>void pal_pe_call_smc(ARM_SMC_ARGS *ArmSmcArgs, int32_t Conduit)</code>	Abstracts the PE wakeup and execute functionality. Ideally, this function calls the PSCI_ON SMC command.
update_elr	<code>void pal_pe_update_elr(void *context, uint64_t offset);</code>	Updates the ELR to return from exception handler to a required address.
get_esr	<code>uint64_t pal_pe_get_esr(void *context);</code>	Returns the exception syndrome from exception handler.
data_cache_ops_by_va	<code>void pal_pe_data_cache_ops_by_va(uint64_t addr, uint32_t type);</code>	Performs cache maintenance operation on an address.
get_far	<code>uint64_t pal_pe_get_far(void *context);</code>	Returns the FAR from exception handler.
install_esr	<code>uint32_t pal_pe_install_esr(uint32_t exception_type, void (*esr)(uint64_t, void *));</code>	Abstracts the exception handler installation steps. The input arguments are exception type and function pointer of the handler that has to be called when the exception of the given type occurs. It returns zero on success and non-zero on failure.
pal_el3_enable_ns_encryption	<code>void pal_el3_enable_ns_encryption(void);</code>	If NS Encryption is programmable then this API must enable NS_Encryption.
pal_el3_disable_ns_encryption	<code>void pal_el3_disable_ns_encryption(void);</code>	If NS Encryption is programmable then this API must disable NS_Encryption.
pal_el3_pas_filter_active_mode	<code>void pal_el3_pas_filter_active_mode(int enable);</code>	This API is used to set or clear the active mode of PAS_FILTER in the system.
pal_el3_prog_legacy_tz	<code>void pal_el3_prog_legacy_tz(int enable);</code>	This API is used to program the LEGACY_TZ input to enable or disable it in the system.
pal_write_reset_status	<code>void pal_write_reset_status(uint64_t nvm_mem, uint32_t status)</code>	This API is used to write the reset status on Non-Volatile memory.
pal_read_reset_status	<code>uint32_t pal_read_reset_status(uint64_t nvm_mem)</code>	This API reads the reset status from Non-Volatile memory.
pal_save_global_test_data	<code>void pal_save_global_test_data(uint64_t nvm_mem, uint32_t total_tests, uint32_t tests_passed, uint32_t tests_failed)</code>	This API saves the test status like total tests, tests passed and tests failed before reset on NV memory.
pal_restore_global_test_data	<code>void pal_restore_global_test_data(uint64_t nvm_mem, uint32_t *total_tests, uint32_t *tests_passed, uint32_t *tests_failed)</code>	This API restores the tests status like total tests, tests passed and tests failed from NV memory after a system reset.

API name	Function prototype	Description
pal_get_cpu_count	uint32_t pal_get_cpu_count(void);	Returns the total number of CPUs or PEs available to PAL.
pal_get_phy_mpidr_list_base	uint64_t *pal_get_phy_mpidr_list_base(void);	Returns the base address of the physical MPIDR list.
pal_pe_get_elr	uint64_t pal_pe_get_elr(void *context);	Returns the ELR from the PE exception context.
pal_psci_get_conduit	int32_t pal_psci_get_conduit(void);	Returns the PSCI conduit that the platform uses (SMC or HVC).

Each PE information entry structure can hold information for a PE in the system. The types of information are:



Note

```
typedef struct {
    UINT32    pe_num;           /* PE Index */
    UINT32    attr;            /* PE attributes */
    UINT64    mpidr;           /* PE MPIDR */
    UINT32    pmu_gsicv;        /* PMU Interrupt */
    UINT32    gmain_gsicv;      /* GIC Maintenance Interrupt */
    UINT32    acpi_proc_uid;    /* ACPI Processor UID */
    UINT32    level_1_res[MAX_L1_CACHE_RES]; /* index of level 1 cache(s) in
    cache_info_table */
} PE_INFO_ENTRY;
```

3.2.3 GIC APIs

These APIs provide the information and functionality required by the test suite that accesses features of a GIC.

Table 3-3: GIC APIs and their descriptions

API name	Function prototype	Description
create_info_table	void pal_gic_create_info_table(GIC_INFO_TABLE *gic_info_table);	Gathers information about the GIC sub-system and fills the gic_info_table with the relevant data.
install_isr	uint32_t pal_gic_install_isr(uint32_t int_id, void (*isr)(void));	Abstracts the steps required to register an interrupt handler to an IRQ number. It also enables the interrupt in the GIC CPU interface and Distributor. It returns 0 on success and -1 on failure.
end_of_interrupt	uint32_t pal_gic_end_of_interrupt(uint32_t int_id);	Indicates completion of interrupt processing by writing to the end of interrupt register in the GIC CPU interface. It returns 0 on success and -1 on failure.
request_irq	uint32_t pal_gic_request_irq(unsigned int irq_num, unsigned int mapped_irq_num, void *isr);	Registers the interrupt handler for a given IRQ. irq_num: hardware IRQ number mapped_irq_num: mapped IRQ number isr: Interrupt Service Routine that returns the status
free_irq	void pal_gic_free_irq(unsigned int irq_num, unsigned int mapped_irq_num);	Frees the registered interrupt handler for a given IRQ. irq_num: hardware IRQ number mapped_irq_num: mapped IRQ number
set_intr_trigger	uint32_t pal_gic_set_intr_trigger(uint32_t int_id, INTR_TRIGGER_INFO_TYPE_e trigger_type);	Sets the trigger type to edge or level. int_id: interrupt ID which must be enabled and the service routine installed for trigger_type: interrupt trigger type edge or level

- Each GIC information entry structure can hold information for any of the seven types of GIC components. The seven types of entries are:

```
typedef enum {
    ENTRY_TYPE_CPUIF = 0x1000,
    ENTRY_TYPE_GICD,
    ENTRY_TYPE_GICC_GICRD,
    ENTRY_TYPE_GICR_GICRD,
    ENTRY_TYPE_GICITS,
    ENTRY_TYPE_GIC_MSI_FRAME,
    ENTRY_TYPE_GICH
}GIC_INFO_TYPE_e;
```



- In addition to the type, each entry contains the base address of each type, entry_id for entry type ITS, and length in case of Redistributor range address length.

```
typedef struct {
    UINT32 type;
    UINT64 base;
    UINT32 entry_id;
    UINT64 length;
    UINT32 flags;
    UINT32 spi_count;
    UINT32 spi_base;
}GIC_INFO_ENTRY;
```

3.2.4 PCIe APIs

These APIs provide the information and functionality required by the test suite that accesses features of PCIe subsystem.

Table 3-4: PCIe APIs and their descriptions

API name	Function prototype	Description
create_info_table	void pal_pcie_create_info_table(PCIE_INFO_TABLE *PcieTable);	Abstracts the steps to gather PCIe information in the system and fills the PCIe info_table. Ideally, this function reads the ACPI MCFG table to retrieve the ECAM base address.
enumerate	void pal_pcie_enumerate(void);	Performs the PCIe enumeration.
io_read_cfg	uint32_t pal_pcie_io_read_cfg(uint32_t bdf, uint32_t offset, uint32_t *data);	Abstracts the configuration space read of a device identified by Bus, Device, and Function (BDF). This is used only in peripheral tests and need not be implemented in Linux. It returns either success or failure. bdf: PCI Bus, Dev, and Func offset: Offset in the configuration space from where data is to be read data: Stores the value read from the configuration space

API name	Function prototype	Description
io_write_cfg	<code>void pal_pcie_io_write_cfg(uint32 bdf, uint32 offset, uint32 data);</code>	Abstracts the configuration space write of a device identified by BDF (Bus, Device, and Function). Writes 32-bit data to the configuration space of the device at an offset. bdf: PCI Bus, Dev, and Func offset: Offset in the configuration space from where data is to be read data: Stores the value read from the configuration space
get_mcfg_ecam	<code>uint64_t pal_pcie_get_mcfg_ecam();</code>	Returns the PCI ECAM address from the ACPI MCFG table address.
get_msi_vectors	<code>uint32_t pal_get_msi_vectors(uint32_t seg, uint32_t bus, uint32_t dev, uint32_t fn, PERIPHERAL_VECTOR_LIST **mvector);</code>	Creates a list of MSI(X) vectors for a device. It returns the number of MSI(X) vectors. seg: PCI segment number bus: PCI bus number dev: PCI device number fn: PCI function number mvector: Pointer to MSI(X) address
scan_bridge_devices_and_check_memtype	<code>uint32_t pal_pcie_scan_bridge_devices_and_check_memtype (uint32_t seg, uint32_t bus, uint32_t dev, uint32_t fn);</code>	Scans the bridge devices and checks the memory type. seg: PCI segment number bus: PCI bus number dev: PCI device number fn: PCI function number
get_pcie_type	<code>uint32_t pal_pcie_get_pcie_type(uint32_t seg, uint32_t bus, uint32_t dev, uint32_t fn);</code>	Gets the PCIe device or port type. seg: PCI segment number bus: PCI bus number dev: PCI device number fn: PCI function number
p2p_support	<code>uint32_t pal_pcie_p2p_support();</code>	Checks P2P support in the PCIe hierarchy. Returns 1 if P2P feature is not supported and 0 if it is supported.

API name	Function prototype	Description
dev_p2p_support	<code>uint32_t pal_pcie_dev_p2p_support(uint32_t seg, uint32_t bus, uint32_t dev, uint32_t fn);</code>	Checks the PCIe device P2P support. seg: PCI segment number bus: PCI bus number dev: PCI device number fn: PCI function number Returns 1 if P2P feature is not supported, else 0.
is_cache_present	<code>uint32_t pal_pcie_is_cache_present (uint32_t seg, uint32_t bus, uint32_t dev, uint32_t fn);</code>	Checks whether the PCIe device has an Address Translation Cache (ATC). seg: PCI segment number bus: PCI bus number dev: PCI device number fn: PCI function number Returns 0 if the device does not have ATC, else 1.
is_onchip_peripheral	<code>uint32_t pal_pcie_is_onchip_peripheral(uint32_t bdf);</code>	Checks if a PCIe function is an on-chip peripheral. bdf: Segment, PCI Bus, Device, and Function. Returns 1 if the PCIe function is an on-chip peripheral, else 0.
check_device_list	<code>uint32_t pal_pcie_check_device_list(void);</code>	Checks if the PCIe hierarchy matches with the topology described in the information table. Returns 0 if device entries match, else 1.
check_device_valid	<code>uint32_t pal_pcie_check_device_valid(uint32_t bdf);</code>	This API is used as a placeholder to check if the bdf obtained is valid or not. bdf: PCI Seg, bus, device, and function
get_rp_transaction_frwd_support	<code>uint32_t pal_pcie_get_rp_transaction_frwd_support(uint32_t seg, uint32_t bus, uint32_t dev, uint32_t fn);</code>	Gets Root Port (RP) transaction forwarding support. seg: PCI segment number bus: PCI bus number dev: PCI device number fn: PCI function number Returns 0 if RP is not involved in transaction forwarding, else 1.

API name	Function prototype	Description
read_ext_cap_word	<pre>void pal_pcie_read_ext_cap_word(uint32_t seg, uint32_t bus, uint32_t dev, uint32_t fn, uint32_t ext_cap_id, uint8_t offset, uint16_t *val);</pre>	Reads the extended PCIe configuration space at an offset for a capability. seg: PCI segment number bus: PCI bus number dev: PCI device number fn: PCI function number ext_cap_id: PCI capability ID offset: offset of the word in the capability configuration space val: return value
get_bdf_wrapper	<pre>uint32 pal_pcie_get_bdf_wrapper (uint32 ClassCode, uint32 StartBdf);</pre>	Returns the Bus, Device, and Function for a matching class code. ClassCode: 32-bit value of format $\text{ClassCode} \ll 16 \mid \text{sub_class_code}$ StartBdf: 0: start enumeration from host bridge. 1: start enumeration from the input segment, Bus, Device. This is needed since multiple controllers with the same class code are potentially present in a system.
bdf_to_dev	<pre>void *pal_pci_bdf_to_dev(uint32_t bdf);</pre>	Returns the PCI device structure for the given bdf. bdf: PCI Bus, Device, and Function.
read_config_byte	<pre>void pal_pci_read_config_byte(uint32_t bdf, uint8_t offset, uint8_t *val);</pre>	Reads one byte from the PCI configuration space for the current BDF at given offset. bdf: PCI Bus, Device, and Function offset: offset in the PCI configuration space for that BDF val: return value
write_config_byte	<pre>void pal_pci_write_config_byte(uint32_t bdf, uint8_t offset, uint8_t val);</pre>	Writes one byte from the PCI configuration space for the current BDF at a given offset. bdf: PCI Bus, Device, and Function offset: offset in the PCI configuration space for that BDF val: return value

API name	Function prototype	Description
mem_get_offset	uint32_t pal_pcie_mem_get_offset(uint32_t type);	Returns the memory offset that can be accessed from the BAR base. t type: Size of the offset required
pal_pci_cfg_write	void pal_pci_cfg_write(uint32_t bus, uint32_t dev, uint32_t func, int offset, int data);	Writes to the PCI configuration space using bus or dev function.
pal_pci_cfg_read	uint32_t pal_pci_cfg_read(uint32_t bus, uint32_t dev, uint32_t func, int offset, uint32_t *value);	Reads from the PCI configuration space using bus or dev function.
pal_pcie_program_bar_reg	void pal_pcie_program_bar_reg(uint32_t bus, uint32_t dev, uint32_t func);	Programs BAR registers for the function.
pal_pcie_enumerate_device	uint32_t pal_pcie_enumerate_device(uint32_t bus, uint32_t sec_bus);	Enumerates PCIe devices or bridges.
pal_pcie_bar_mem_read	uint32_t pal_pcie_bar_mem_read(uint32_t bdf, uint64_t address, uint32_t *data);	Reads from PCIe BAR memory space.
pal_pcie_bar_mem_write	uint32_t pal_pcie_bar_mem_write(uint32_t bdf, uint64_t address, uint32_t data);	Writes to PCIe BAR memory space.
pal_pcie_get_legacy_irq_map	uint32_t pal_pcie_get_legacy_irq_map(uint32_t seg, uint32_t bus, uint32_t dev, uint32_t fn, PERIPHERAL_IRQ_MAP *irq_map);	Returns the legacy IRQ routing map.
pal_pcie_is_device_behind_smmu	uint32_t pal_pcie_is_device_behind_smmu(uint32_t seg, uint32_t bus, uint32_t dev, uint32_t fn);	Checks if the device path is behind SMMU.
pal_pcie_get_root_port_bdf	uint32_t pal_pcie_get_root_port_bdf(uint32_t *seg, uint32_t *bus, uint32_t *dev, uint32_t *func);	Returns the root port BDF.
pal_pcie_get_device_type	uint32_t pal_pcie_get_device_type(uint32_t seg, uint32_t bus, uint32_t dev, uint32_t fn);	Returns the PCIe device type.
pal_pcie_get_snoop_bit	uint32_t pal_pcie_get_snoop_bit(uint32_t seg, uint32_t bus, uint32_t dev, uint32_t fn);	Returns the snoop or no-snoop bit state.
pal_pcie_get_dma_support	uint32_t pal_pcie_get_dma_support(uint32_t seg, uint32_t bus, uint32_t dev, uint32_t fn);	Reports the DMA support.
pal_pcie_get_dma_coherent	uint32_t pal_pcie_get_dma_coherent(uint32_t seg, uint32_t bus, uint32_t dev, uint32_t fn);	Reports the DMA coherency support or state.
pal_pcie_is_devicedma_64bit	uint32_t pal_pcie_is_devicedma_64bit(uint32_t seg, uint32_t bus, uint32_t dev, uint32_t fn);	Reports 64-bit DMA addressing support.
pal_pcie_device_driver_present	uint32_t pal_pcie_device_driver_present(uint32_t seg, uint32_t bus, uint32_t dev, uint32_t fn);	Indicates the driver presence or binding.
pal_link_get_exposure	uint32_t pal_link_get_exposure(uint16_t segment, uint8_t bus, uint8_t device, uint8_t function, uint32_t *exposed);	Returns the link exposure or security status.

This data structure holds the PCIe subsystem information.



Note

```
/**
@brief PCI Express Info Table
**/
typedef struct {
    addr_t ecam_base;           ///< ECAM Base address
    uint32_t segment_num;       ///< Segment number of this ECAM
    uint32_t start_bus_num;     ///< Start Bus number for this ecam space
    uint32_t end_bus_num;       ///< Last Bus number
}
```

```
} PCIE_INFO_BLOCK;
```

The data structure is repeated for the number of ECAM ranges in the system.

```
typedef struct {
    uint32_t num_entries;
    PCIE_INFO_BLOCK block[];
} PCIE_INFO_TABLE;
```

3.2.5 SMMU APIs

These functions abstract information that is specific to the operations of the SMMUs in the system.

Table 3-5: SMMU APIs and their descriptions

API name	Function prototype	Description
create_info_table	<code>void pal_smmu_create_info_table(SMMU_INFO_TABLE *smmu_info_table);</code>	Abstracts the steps to gather information about SMMUs in the system and fills the info_table.
check_device_iova	<code>uint32_t pal_smmu_check_device_iova(void *port, uint64_t dma_addr);</code>	Checks if the input DMA address belongs to the input device. This can be done by keeping track of the DMA addresses generated by the device using the start and stop monitor calls defined below or by reading the IOVA table of the device and looking for the input address. 0 is returned if address belongs to the device. Non-zero is returned if there are IMPLEMENTATION DEFINED error values.
device_start_monitor_iova	<code>void pal_smmu_device_start_monitor_iova(void *port);</code>	A hook to start the process of saving DMA addresses being used by the input device. It is used by the test to indicate the upcoming DMA transfers to be recorded and the test queries for the address through the check_device_iova call.
device_stop_monitor_iova	<code>void pal_smmu_device_stop_monitor_iova(void *port);</code>	Stops the recording of the DMA addresses being used by the input port.
max_pasids	<code>uint32_t pal_smmu_max_pasids(uint64_t smmu_base);</code>	Returns the maximum PASID value supported by the SMMU controller. For SMMUV3, this value can be read from the IDR1 register. 0 is returned when PASID support is not detected. Non-zero is returned if maximum PASID value supported for the input SMMU.
pa2iova	<code>uint64_t pal_smmu_pa2iova(uint64_t SmmuBase, uint64_t Pa);</code>	Converts physical address to I/O virtual address. SmmuBase: physical address of the SMMU for conversion to virtual address. Pa: physical address to use in conversion. Returns 0 on success and 1 on failure.
smmu_disable	<code>uint32_t pal_smmu_disable(uint64_t SmmuBase);</code>	Globally disables the SMMU based on input base address. SmmuBase: physical address of the SMMU that needs to be globally disabled. Returns 0 for success and 1 for failure.
create_pasid_entry	<code>uint32_t pal_smmu_create_pasid_entry(uint64_t smmu_base, uint32_t pasid);</code>	Prepares the SMMU page tables to support input PASID. smmu_base: physical address of the SMMU for which PASID support is needed. pasid: Process Address Space Identifier. Returns 0 for success and 1 for failure.

API name	Function prototype	Description
pal_iovirt_create_info_table	void pal_iovirt_create_info_table(IOVIRT_INFO_TABLE *iovirt);	Builds the IOVIRT topology information table.
pal_iovirt_get_rc_smmu_base	uint64_t pal_iovirt_get_rc_smmu_base(IOVIRT_INFO_TABLE *iovirt, uint32_t rc_seg_num, uint32_t rid);	Returns the SMMU base for the RC segment and RID mapping.

3.2.6 Exerciser APIs

These APIs abstract information specific to the operations of PCIe stimulus generation hardware.

Table 3-6: Exerciser APIs and descriptions

API Name	Function prototype	Description
set_param	uint32_t pal_exerciser_set_param(EXERCISER_PARAM_TYPE type, uint64_t value1, uint64_t value2, uint32_t instance)	Writes the configuration parameters to the PCIe stimulus generation hardware indicated by the instance number. The supported configuration parameters include: 1 – SNOOP_ATTRIBUTES 2 – LEGACY_IRQ 3 – DMA_ATTRIBUTES 4 – P2P_ATTRIBUTES 5 – PASID_ATTRIBUTES 6 – MSIX_ATTRIBUTES 7 – CFG_TXN_ATTRIBUTES 8 – ERROR_INJECT_TYPE 9 – ATS_RES_ATTRIBUTES 10 – TRANSACTION_TYPE 11 – NUM_TRANSACTIONS 12 – ADDRESS_ATTRIBUTES 13 – DATA_ATTRIBUTES 14 – ERROR_INJECT_TYPE 15 – EXERCISER_CXL_CMD_OP 16 – EXERCISER_CXL_CMD_ADDR 17 – EXERCISER_CXL_CMD_DATA0 18 – EXERCISER_SEC_SID value2 is an optional argument and must be ignored for some configuration parameters.

API Name	Function prototype	Description
get_param	uint32_t pal_exerciser_get_param(EXERCISER_PARAM_TYPE type, uint64_t *value1, uint64_t *value2, uint32_t instance)	Returns the requested configuration parameter values through 64-bit input arguments value1 and value2. The function returns a value of 1 to indicate read success and 0 to indicate read failure.
set_state	uint32_t pal_exerciser_set_state(EXERCISER_STATE state, uint64_t *value, uint32_t instance)	Sets the state of the PCIe stimulus generation hardware. The supported states include: 1 – RESET, hardware in reset state. 2 – ON, this state is set after hardware is initialized and is ready to generate stimulus. 3 – OFF, this state is set to indicate that hardware can no longer generate stimulus. 4 – ERROR, this state is set to signal an error with hardware.
get_state	uint32_t pal_exerciser_get_state(EXERCISER_STATE state, uint64_t *value, uint32_t instance)	Returns the state of the PCIe stimulus generation hardware of the requested instance.
ops	uint32_t pal_exerciser_ops(EXERCISER_OPS ops, uint64_t param, uint32_t instance)	Abstracts the steps to implement the requested operation on the PCIe stimulus generation hardware. Following are the supported operations: 1 – START_DMA 2 – GENERATE_MSI 3 – GENERATE_L_INTR 4 – MEM_READ 5 – MEM_WRITE 6 – CLEAR_INTR 7 – PASID_TLP_START 8 – PASID_TLP_STOP 9 – TXN_NO_SNOOP_ENABLE 10 – TXN_NO_SNOOP_DISABLE 11 – START_TXN_MONITOR 12 – STOP_TXN_MONITOR 13 – ATS_TXN_REQ 14 – INJECT_ERROR 15 – ATS_TXN_REQ 16 – ATS_INV_CACHE 17 – CXL_CMD_START
get_data	uint32_t pal_exerciser_get_data(EXERCISER_DATA_TYPE type, exerciser_data_t *data, uint32_t instance)	Returns either the configuration space or the BAR space information depending on the input argument type. The argument type can take one of the following two values: 1 – EXERCISER_DATA_CFG_SPACE 2 – EXERCISER_DATA_BAR0_SPACE

API Name	Function prototype	Description
is_bdf_exerciser	uint32_t pal_is_bdf_exerciser(uint32_t bdf)	Checks if the device is an exerciser. Returns 1 if device is an exerciser, else 0.
get_ecsr_base	uint64_t pal_exerciser_get_ecsr_base(uint32_t Bdf, uint32_t BarIndex)	Returns the ECSR base address of a particular BAR Index.
get_pcie_config_offset	uint64_t pal_exerciser_get_pcie_config_offset(uint32_t Bdf)	Returns the configuration address of the given bdf.
start_dma_direction	uint32_t pal_exerciser_start_dma_direction(uint64_t Base, EXERCISER_DMA_ATTRDirection)	Triggers the DMA operation.
find_pcie_capability	uint32_t pal_exerciser_find_pcie_capability(uint32_t ID, uint32_t Bdf, uint32_t Value, uint32_t *Offset)	Returns 0 if the PCI capability is found.
device_lock	uint32_t pal_device_lock(uint32_t bdf)	Transitions the device TDI state to RUN.
device_unlock	uint32_t pal_device_unlock(uint32_t bdf)	Transitions the TDI from RUN to CONFIG_UNLOCKED state.

3.2.7 CXL APIs

These APIs provide the information and functionality required by the test suite that accesses CXL subsystem features.

Table 3-7: CXL APIs and their descriptions

API name	Function prototype	Description
create_info_table	void pal_cxl_create_info_table(CXL_INFO_TABLE *CxlTable);	Describes the steps to gather CXL host bridge and decoder information and fills the CXL table.
get_cedt_ptr	uint64_t pal_get_cedt_ptr(void);	Returns the pointer or address of the ACPI CEDT table used for CXL discovery.
get_host_bridge_uid	uint32_t pal_cxl_get_host_bridge_uid(uint32_t bdf, uint32_t *uid);	Returns the host bridge UID for the input PCIe BDF. Returns status to indicate success or failure.
root_port_ide_program_and_enable	uint32_t pal_cxl_root_port_ide_program_and_enable(uint32_t rp_bdf, uint8_t stream_id, uint8_t key_slot, const CXL_IDE_KEY_BUFFER *rx_key, const CXL_IDE_KEY_BUFFER *tx_key);	Programs IDE key material for the root port stream and key slot and enables IDE.
root_port_ide_disable	uint32_t pal_cxl_root_port_ide_disable(uint32_t rp_bdf, uint8_t stream_id, uint8_t key_slot);	Disables IDE for the given root port stream and key slot.
rp_is_not_subject_to_host_gpc	uint32_t pal_cxl_rp_is_not_subject_to_host_gpc(uint32_t rp_bdf);	Checks whether the input root port is exempt from host GPC constraints.
is_chi_c2c_supported	uint32_t pal_cxl_is_chi_c2c_supported(uint32_t bdf);	Checks whether CHI C2C capability or support is present for the input CXL path or device.
pal_is_coherent_da_supported	uint32_t pal_is_coherent_da_supported(void);	Returns whether coherent DA is supported.
pal_cxl_root_port_ide_program_and_enable	uint32_t pal_cxl_root_port_ide_program_and_enable(uint64_t bar0_base, uint8_t stream_id, uint8_t key_slot, const CXL_IDE_KEY_BUFFER *rx_key, const CXL_IDE_KEY_BUFFER *tx_key);	EL3 helper that programs IDE and enables it using BAR0-based KM registers.
pal_cxl_root_port_ide_disable	uint32_t pal_cxl_root_port_ide_disable(uint64_t bar0_base, uint8_t stream_id, uint8_t key_slot);	EL3 helper that disables IDE using BAR0-based KM registers.
pal_cxl_root_port_ide_get_km_base	uint32_t pal_cxl_root_port_ide_get_km_base(uint64_t bar0_base, uint64_t *ide_km_base);	EL3 helper that derives the IDE KM base address from BAR0.

The following data structures define CXL subsystem information.

```
typedef struct {
uint32_t uid;
uint32_t component_reg_type;
uint64_t component_reg_base;
uint64_t component_reg_length;
uint32_t hdm_decoder_count;
uint32_t cfmws_count;
uint64_t cfmws_base[CXL_MAX_CFMWS_WINDOWS];
uint64_t cfmws_length[CXL_MAX_CFMWS_WINDOWS];
} CXL_INFO_BLOCK;
```



This data structure is repeated for the number of CXL host bridges.

```
typedef struct {
uint32_t num_entries;
CXL_INFO_BLOCK device[];
} CXL_INFO_TABLE;
```

This data structure holds the IDE Key management buffers.

```
typedef struct {
uint32_t key[8];
uint32_t iv[3];
} CXL_IDE_KEY_BUFFER;
```

3.2.8 Miscellaneous APIs

Miscellaneous APIs are described in the following table.

Table 3-8: Miscellaneous APIs and their descriptions

API name	Function prototype	Description
print	void pal_print(char *string, uint64_t data);	Sends a formatted string to the output console. string: An ASCII string. data: Data for the formatted output.
print_raw	void pal_print_raw(uint64_t addr, char *string, uint64_t data);	Sends a string to the output console without using the platform print function. This function gets COMM port address and directly writes to the address character by character. addr: Address to be written. string: An ASCII string. data: Data for the formatted output.

API name	Function prototype	Description
strncmp	<code>pal_strncmp uint32_t pal_strncmp (char *FirstString, char *SecondString, uint32_t Length);</code>	Compares two strings. Returns zero if strings are identical, else a nonzero value. FirstString: The pointer to the first null-terminated ASCII string. SecondString: The pointer to the second null-terminated ASCII string. Length: The maximum number of ASCII characters for comparison.
mmio_read	<code>uint32 pal_mmio_read(uint64 addr);</code>	Provides a single point of abstraction to read from all memory-mapped I/O addresses. addr: 64-bit input address return: 32-bit data read from the input address
mmio_read8	<code>pal_mmio_read8(uint64 addr);</code>	Provides a single point of abstraction to read 8-bit data from all memory-mapped I/O addresses. addr: 64-bit input address return: 8-bit data read from the input address
mmio_read16	<code>pal_mmio_read16(uint64 addr);</code>	Provides a single point of abstraction to read 16-bit data from all memory-mapped I/O addresses. addr: 64-bit input address return: 16-bit data read from the input address
mmio_read64	<code>pal_mmio_read64(uint64 addr);</code>	Provides a single point of abstraction to read 64-bit data from all memory-mapped I/O addresses. addr: 64-bit input address return: 64-bit data read from the input address
mmio_write	<code>void pal_mmio_write(uint64 addr, uint32 data);</code>	Provides a single point of abstraction to write to all memory-mapped I/O addresses. addr: 64-bit input address data: 32-bit data to write to address
mmio_write8	<code>pal_mmio_write8(uint64 addr, uint8 data);</code>	Provides a single point of abstraction to write 8-bit data to all memory-mapped I/O addresses. addr: 64-bit input address data: 8-bit data to write to address
mmio_write16	<code>pal_mmio_write16(uint64 addr, uint16 data);</code>	Provides a single point of abstraction to write 16-bit data to all memory-mapped I/O addresses. addr: 64-bit input address data: 16-bit data to write to address
mmio_write64	<code>pal_mmio_write(uint64 addr, uint64 data);</code>	Provides a single point of abstraction to write 64-bit data to all memory-mapped I/O addresses. addr: 64-bit input address data: 64-bit data to write to address

API name	Function prototype	Description
mem_free_shared	<code>pal_mem_free_shared(void);</code>	Frees the allocated shared memory region.
mem_get_shared_addr	<code>pal_mem_get_shared_addr(void);</code>	Returns the base address of the shared memory region to the VAL layer.
mem_alloc	<code>void pal_mem_alloc(unsigned int size);</code>	Allocates memory of the requested size. size: size of the memory region to be allocated Returns virtual address on success and null on failure.
mem_calloc	<code>void * pal_mem_calloc(uint32_t num, uint32_t Size);</code>	Allocates requested buffer size in bytes with zeros in a contiguous memory and returns the base address of the range.
mem_allocate_shared	<code>pal_mem_allocate_shared (uint32_t num_pe, uint32_t sizeofentry);</code>	Allocates memory which is to be used to share data across PEs. num_pe: number of PEs in the system sizeofentry: size of memory region allocated to each PE Returns none.
mem_free	<code>void pal_mem_free(void *buffer);</code>	Frees the memory allocated by UEFI framework APIs. buffer: the base address of the memory range to be free
mem_cpy	<code>void *pal_memcpy(void *dest_buffer, void *src_buffer, uint32_t len);:</code> base address of the memory range to be freed	Copies a source buffer to a destination buffer and returns the destination buffer. dest_buffer: pointer to the destination buffer of the memory copy src_buffer: pointer to the source buffer of the memory copy len: number of bytes to copy from source buffer to destination buffer Returns the destination buffer.
mem_compare	<code>uint32 pal_mem_compare(void *src, void *dest, uint32 len);</code>	Compares the contents of the source and destination buffers. src: base address of the memory, source buffer to be compared dest: destination buffer to be compared with len: length of the comparison to be performed
mem_alloc_cacheable	<code>void pal_mem_alloc_cacheable(uint32_t bdf, uint32_t size, void *pa);</code>	Allocates cacheable memory of the requested size. bdf: BDF of the requesting PCIe device size: size of the memory region to be allocated pa: physical address of the allocated memory

API name	Function prototype	Description
mem_free_cacheable	<code>void pal_mem_free_cacheable(uint32_t bdf, uint32_t size, void *va, void *pa);</code>	Frees the cacheable memory allocated by Linux DMA Framework APIs. bdf: Bus, Device, and Function of the requesting PCIe device size: size of memory region to be freed va: virtual address of the memory to be freed pa: physical address of the memory to be freed
mem_virt_to_phys	<code>void pal_mem_virt_to_phys(void *va);</code>	Returns the physical address of the input virtual address. va: virtual address of the memory to be converted Returns the physical address.
time_delay_ms	<code>uint64 pal_time_delay_ms (uint64 MicroSeconds);</code>	Stalls the CPU for the specified number of microseconds. MicroSeconds: the minimum number of microseconds to be delayed Returns the value of the microseconds given as input.
mem_set	<code>void pal_mem_set (void *buf, uint32 size, uint8 value);</code>	A buffer with a known specified input value. buf: pointer to the buffer to fill size: number of bytes in the buffer to fill value: value to fill the buffer with
page_size	<code>uint32_t pal_mem_page_size();</code>	Returns the memory page size (in bytes) used by the platform.
alloc_pages	<code>void* pal_mem_alloc_pages (uint32 NumPages);</code>	Allocates the requested number of memory pages.
free_pages	<code>void pal_mem_free_pages (void *PageBase, uint32_t NumPages);</code>	Frees pages as requested.
phys_to_virt	<code>void* pal_mem_phys_to_virt (uint64_t Pa);</code>	Returns the VA of the input PA. Pa: Physical Address of the memory to be converted. Returns the VA.
target_is_bm	<code>uint32_t pal_target_is_bm();</code>	Checks if the system information is passed using bare-metal.
aligned_alloc	<code>void *pal_aligned_alloc(uint32_t alignment, uint32_t size);</code>	Allocates memory with the given alignment. alignment: Specifies the alignment. size: Requested memory allocation size. Returns pointer to the allocated memory with requested alignment.
mem_alloc_at_address	<code>void *pal_mem_alloc_at_address(uint64_t mem_base, uint64_t size);</code>	Allocates memory in the given memory base.
mem_free_at_address	<code>void pal_mem_free_at_address(uint64_t mem_base, uint64_t size);</code>	Frees the allocated memory in the given memory base.
pal_timer_create_info_table	<code>void pal_timer_create_info_table(TIMER_INFO_TABLE *timer_info_table);</code>	Builds the timer information table.

API name	Function prototype	Description
pal_peripheral_create_info_table	<code>void pal_peripheral_create_info_table(PERIPHERAL_INFO_TABLE *per_info_table);</code>	Builds the peripheral information table.
pal_register_get_num_entries	<code>uint32_t pal_register_get_num_entries(void);</code>	Returns the number of register entries.
pal_register_create_info_table	<code>void pal_register_create_info_table(REGISTER_INFO_TABLE *registerInfoTable);</code>	Builds the register information table.
pal_root_register_create_info_table	<code>void pal_root_register_create_info_table(ROOT_REGSTR_TABLE *table);</code>	Builds the root register information table.
pal_mem_region_create_info_table	<code>void pal_mem_region_create_info_table(MEM_REGN_INFO_TABLE *gpc_table, MEM_REGN_INFO_TABLE *pas_table);</code>	Builds memory region information table.
pal_uart_print	<code>void pal_uart_print(int log, const char *fmt, ...);</code>	Provides UART print helper for logging.
pal_mmu_add_mmap	<code>void pal_mmu_add_mmap(void);</code>	Adds PAL MMU mappings.
pal_mmu_get_mmap_list	<code>void *pal_mmu_get_mmap_list(void);</code>	Returns the PAL MMU mapping list base.
pal_mmu_get_mapping_count	<code>uint32_t pal_mmu_get_mapping_count(void);</code>	Returns the number of PAL MMU mappings.
pal_is_legacy_tz_enabled	<code>uint32_t pal_is_legacy_tz_enabled(void);</code>	Returns the LEGACY_TZ state.
pal_is_ns_encryption_programmable	<code>uint32_t pal_is_ns_encryption_programmable(void);</code>	Returns the NS-encryption programmability status.
pal_is_pas_filter_mode_programmable	<code>uint32_t pal_is_pas_filter_mode_programmable(void);</code>	Returns the PAS-filter programmability status.

Proprietary Notice

This document is CONFIDENTIAL and any use by you is subject to the terms of the agreement between you and Arm Limited ("Arm") or the terms of the agreement between you and the party authorized by Arm to disclose this document to you.

This document is protected by copyright and other related rights and the use or implementation of the information contained in this document may be protected by one or more patents or pending patent applications. No part of this document may be reproduced in any form by any means without the express prior written permission of Arm. No license, express or implied, by estoppel or otherwise to any intellectual property rights is granted by this document unless specifically stated.

Your access to the information in this document is conditional upon your acceptance that, without obtaining Arm's prior written consent, you will not use or permit others to use the information: (i) for the purposes of determining whether the subject matter of this document infringes any third party patents; (ii) for developing technology or products which avoid any of Arm's intellectual property; (iii) as a reference for modifying existing patents or patent applications or creating any continuation, continuation in part, or extension of existing patents or patent applications; or (iv) for generating data for publication or disclosure to third parties, which compares the performance or functionality of the Arm technology described in this document with any other products created by you or a third party.

The content of this document is informational only. Any solutions presented herein are subject to changing conditions, information, scope, and data. This document was produced using reasonable efforts based on information available as of the date of issue of this document. The scope of information in this document may exceed that which Arm is required to provide, and such additional information is merely intended to further assist the recipient and does not represent Arm's view of the scope of its obligations. You acknowledge and agree that you possess the necessary expertise in system security and functional safety and that you shall be solely responsible for compliance with all legal, regulatory, safety and security related requirements concerning your products, notwithstanding any information or support that may be provided by Arm herein. In addition, you are responsible for any applications which are used in conjunction with any Arm technology described in this document, and to minimize risks, adequate design and operating safeguards should be provided for by you.

This document may include technical inaccuracies or typographical errors. THIS DOCUMENT IS PROVIDED "AS IS". ARM PROVIDES NO REPRESENTATIONS AND NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE WITH RESPECT TO THE DOCUMENT. For the avoidance of doubt, Arm makes no representation with respect to, and has undertaken no analysis to identify or understand the scope and content of, any patents, copyrights, trade secrets, trademarks, or other rights.

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL ARM BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF ARM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Reference by Arm to any third party's products or services within this document is not an express or implied approval or endorsement of the use thereof.

This document consists solely of commercial items. You shall be responsible for ensuring that any permitted use, duplication, or disclosure of this document complies fully with any relevant export laws and regulations to assure

that this document or any portion thereof is not exported, directly or indirectly, in violation of such export laws. Use of the word “partner” in reference to Arm’s customers is not intended to create or refer to any partnership relationship with any other company. Arm may make changes to this document at any time and without notice.

If any of the provisions contained in these terms conflict with any of the provisions of any click through or signed written agreement covering this document with Arm, then the click through or signed written agreement prevails over and supersedes the conflicting provisions of these terms. This document may be translated into other languages for convenience, and you agree that if there is any conflict between the English version of this document and any translation, the terms of the English version of this document shall prevail.

The validity, construction and performance of this notice shall be governed by English Law.

The Arm corporate logo and words marked with ® or ™ are registered trademarks or trademarks of Arm Limited (or its affiliates) in the US and/or elsewhere. Please follow Arm’s trademark usage guidelines at <https://www.arm.com/company/policies/trademarks>. All rights reserved. Other brands and names mentioned in this document may be the trademarks of their respective owners.

Arm Limited. Company 02557590 registered in England.

110 Fulbourn Road, Cambridge, England CB1 9NJ.

PRE-1122-V1.0

Product and document information

Read the information in these sections to understand the release status of the product and documentation, and the conventions used in the Arm documents.

Product status

All products and Services provided by Arm require deliverables to be prepared and made available at different levels of completeness. The information in this document indicates the appropriate level of completeness for the associated deliverables.

Product completeness status

The information in this document is for an Alpha product, that is a product under development.

Revision history

These sections can help you understand how the document has changed over time.

Document release information

The Document history table gives the issue number and the released date for each released issue of this document.

Document history

Issue	Date	Confidentiality	Change
0300-04	20 July 2026	Non-Confidential	Alpha v0.6 release for issue C.a
0300-03	18 March 2026	Non-Confidential	Alpha release for issue C.a
0300-02	22 April 2025	Non-Confidential	EAC release for issue B.a
0300-01	22 January 2025	Non-Confidential	EAC release
0200-01	15 November 2024	Non-Confidential	Beta release for issue B
0100-02	9 August 2024	Non-Confidential	First internal release
0100-01	13 December 2023	Non-Confidential	First internal for v1.0
0007-01	6 November 2023	Non-Confidential	First release for v0.7
0006-01	25 August 2023	Confidential	First internal release for v0.6
0005-01	24 April 2023	Confidential	First internal release for v0.5

The Change history tables describe the technical changes between released issues of this document in reverse order. Issue numbers match the revision history in [Document release information](#) on page 33.

Table 2: Difference between Issue 0300-03 and Issue 0300-04

Change	Location
Updated the note for pal_el3 configuration file.	See 2.2.1 Source code directory on page 11

Table 3: Differences between Issue 0300-02 and Issue 0300-03

Change	Location
Added new abbreviations.	See 1.1 Abbreviations on page 5
Added a new layer in the compliance test layers table.	See 1.3 Layered software stack on page 6
Added new source code directories and updated the note.	See 2.2.1 Source code directory on page 11
Updated the prerequisites section.	See 2.3.1 Prerequisites on page 12
Updated the EL3 structure and components section.	See 2.3.2 EL3 structure and components on page 13
Added CXL, CDA, and TDISP in the modules and corresponding API names table.	See 3.2.1 API naming convention on page 14
Updated the PE APIs and their descriptions table.	See 3.2.2 PE APIs on page 15
Added new APIs in the PCIe APIs and their descriptions table.	See 3.2.4 PCIe APIs on page 17
Added new APIs in the SMMU APIs and their descriptions table.	See 3.2.5 SMMU APIs on page 22
- Added new supported configuration parameters for set_param. - Added new supported operations for ops.	See 3.2.6 Exerciser APIs on page 23
Added a new section for CXL APIs .	See 3.2.7 CXL APIs on page 25
Added new APIs in the Miscellaneous APIs and their descriptions.	See 3.2.8 Miscellaneous APIs on page 26

Table 4: Differences between Issue 0300-01 and Issue 0300-02

Change	Location
Added new abbreviations.	See 1.1 Abbreviations on page 5
Added DPT and MEC modules.	See 3.2.1 API naming convention on page 14

Table 5: Differences between Issue 0200-01 and Issue 0300-01

Change	Location
No technical changes.	-

Table 6: Differences between Issue 0100-02 and Issue 0200-01

Change	Location
Added new abbreviation.	See 1.1 Abbreviations on page 5.
Added new APIs in the exerciser APIs and descriptions table.	See 3.2.6 Exerciser APIs on page 23.
Removed the section GIC ITS.	-
Removed the RME system directory structure image.	See 2.2.1 Source code directory on page 11.

Table 7: Differences between Issue 0100-01 and Issue 0100-02

Change	Location
Added new abbreviations.	See 1.1 Abbreviations on page 5.
Updated the description.	See 1.2 RME System ACS on page 6 .
Removed the sections compliance test and test platform abstraction.	See 1. Overview to RME System ACS on page 5.
Updated the section coding guidelines to test and infrastructure guidelines.	See 1.3.2 Test and infrastructure guidelines on page 8.
Updated the system directory structure.	See 2.2.1 Source code directory on page 11.
Added a module, DA.	See 2.2.1 Source code directory on page 11.

Table 8: Differences between Issue 0007-01 and Issue 0100-01

Change	Location
Updated the prerequisites and added a note.	See 2.3.1 Prerequisites on page 12.
Updated the EL3 code.	See 2.3.2 EL3 structure and components on page 13.

Table 9: Differences between Issue 0006-01 and Issue 0007-01

Change	Location
Added new module and API.	See 3.2.1 API naming convention on page 14.
Added new PE APIs.	See 3.2.2 PE APIs on page 15.

Table 10: Differences between Issue 0006-01 Issue 0007-01

Change	Location
Added new module and API.	See 3.2.1 API naming convention on page 14.
Added new PE APIs.	See 3.2.2 PE APIs on page 15.

Table 11: Differences between Issue 0005-01 and Issue 0006-01

Change	Location
Added new PE APIs.	See 3.2.2 PE APIs on page 15.

Table 12: Issue 0005-01

Change	Location
First release.	-

Conventions

The following subsections describe conventions used in Arm documents.

Glossary

The Arm Glossary is a list of terms used in Arm documentation, together with definitions for those terms. The Arm Glossary does not contain terms that are industry standard unless the Arm meaning differs from the generally accepted meaning.

See the Arm Glossary for more information: developer.arm.com/glossary.

Typographic conventions

Arm documentation uses typographical conventions to convey specific meaning.

Convention	Use
<i>italic</i>	Citations.
bold	Interface elements, such as menu names. Terms in descriptive lists, where appropriate.
monospace	Text that you can enter at the keyboard, such as commands, file and program names, and source code.
monospace <u>underline</u>	A permitted abbreviation for a command or option. You can enter the underlined text instead of the full command or option name.
<and>	Encloses replaceable terms for assembler syntax where they appear in code or code fragments. For example: MRC p15, 0, <Rd>, <CRn>, <CRm>, <Opcode_2>
SMALL CAPITALS	Terms that have specific technical meanings as defined in the <i>Arm® Glossary</i> . For example, IMPLEMENTATION DEFINED , IMPLEMENTATION SPECIFIC , UNKNOWN , and UNPREDICTABLE .



We recommend the following. If you do not follow these recommendations your system might not work.



Your system requires the following. If you do not follow these requirements your system will not work.



You are at risk of causing permanent damage to your system or your equipment, or of harming yourself.



This information is important and needs your attention.



This information might help you perform a task in an easier, better, or faster way.



This information reminds you of something important relating to the current content.

Useful resources

This document contains information that is specific to this product. See the following resources for other useful information.

Arm documents are available on developer.arm.com/documentation.

Confidential documents are only available to licensees, when logged in. Each document link in the following tables provides direct access to the online version of the document.

Arm product resources	Document ID	Confidentiality
Arm® Realm Management Extension (RME) System Architecture	DEN0129	Non-Confidential
Arm® System Memory Management Unit Architecture Specification	IHI0070	Non-Confidential

Arm architecture and specifications	Document ID	Confidentiality
Arm® Architecture Reference Manual for A-profile architecture	DDI0487	Non-Confidential
Arm® Generic Interrupt Controller Architecture Specification for GIC architecture version 3.0 and version 4.0	IHI0069	Non-Confidential